

# IntSeqBERT: Learning Arithmetic Structure in OEIS via Modulo-Spectrum Embeddings

---

Kazuhisa Nakasho

CICM 2026

Iwate Prefectural University, Japan

Code & data: [github.com/aabaa/IntSeqBERT](https://github.com/aabaa/IntSeqBERT)

Today I will introduce IntSeqBERT, a model that learns arithmetic patterns in integer sequences.

# Why Finding Good Problems Matters

## A recent Mathstodon thread by Terence Tao

8 September 2026

Summary of the thread, with context from his earlier posts on AI evaluation.

- Terence Tao worries that AI may use up good open problems as benchmarks without helping us create new ones.
- Studying good problems helps us **understand more and find connections to other questions**.
- He warns that focusing only on answers may harm this process and make it harder to develop **new ideas and good problems**.

*“reveal any further insights or connections to other questions”*

His example: there are many possible questions, but few useful ones, just as there is plenty of sea water but not always enough drinking water.

Sources (paraphrased above; excerpt from post 1/4):

Mathstodon, 8 September 2026: fruitful open problems, insights, and the research ecosystem (thread 1–4).

Related thread: AI evaluation and open problems as a non-renewable resource.

# Long-Term Goal — AI for Mathematical Discovery

- Our goal is to help find new problems by looking for connections in numerical patterns.
  - McKay noticed  $196,884 = 196,883 + 1$ ; this led to *Monstrous Moonshine*.
  - The **Taniyama–Shimura conjecture** links elliptic curves and modular forms. This link was key to the proof of **Fermat's Last Theorem**. Wiles (1995): the semistable case.
  - The BSD conjecture and mirror symmetry also began with numerical patterns (317,206,375 rational curves in the mirror symmetry example).
- AI now takes part in this kind of discovery. Examples include **AlphaProof**, the **Ramanujan Machine**, and **FunSearch**.
- We want to find sequences with similar arithmetic patterns and use those similarities to suggest conjectures. The sequences may come from different fields of mathematics.
- We use the **OEIS**, the Online Encyclopedia of Integer Sequences. Our snapshot contains 391,710 sequences.
- To work towards that goal, we first need a model that captures arithmetic structure. We then measure how much structure it learns.

# Multiplication — Theory and Practice

For such a model, multiplication is a challenge: simple in theory, but hard to learn.

| Setting and study                                                            | Main result                                                                                                                                                     |
|------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Theory: threshold circuits</b><br>Siu & Roychowdhury (1994)               | In theory, multiplication can be implemented with just <b>3 layers</b> .<br>Polynomial-size threshold circuits; depth 3 is optimal in this class.               |
| <b>Practice: trained Transformer</b><br>Shen et al. (2023), Sec. 2 / Table 5 | In practice, even a <b>12-layer model</b> got only <b>84%</b> of 15-digit multiplications right.<br>15×15-digit task; zero-padding and reversed product digits. |
| <b>GPT-4: decimal multiplication</b><br>Dziri et al. (2023)                  | Accuracy falls from <b>59%</b> for 3×3-digit products to <b>4%</b> for 4×4-digit products.                                                                      |
| <b>2-layer Transformer: mod <math>p</math></b><br>Power et al. (2022)        | Generalisation appears only after prolonged training ( <b>grokking</b> ).                                                                                       |

Different model classes and tasks; the experiment does not establish a minimum depth.

Having enough layers does not guarantee that the model will learn multiplication.

For OEIS sequences, input representation is another challenge: a fixed vocabulary cannot cover all integers. Token IDs hide arithmetic structure; out-of-vocabulary values share UNK. OEIS values can reach  $10^{210}$ .

## Key Idea — Show Multiplicative Structure in the Input

- To address these challenges, we represent each integer by its remainders under several moduli. These remainders, or residues, **preserve addition and multiplication**. Reduction mod  $m$  is a **ring homomorphism**:  $\mathbb{Z} \rightarrow \mathbb{Z}/m\mathbb{Z}$ .
- The residues stay small, even when the original values are very large. For  $10^{210}$ , a residue is still in  $\{0, \dots, m-1\}$ . The sequence  $(x_i \bmod m)$  keeps multiplicative structure.
- Classical number theory explains these **periodic patterns**:
  - **Fermat's little theorem** describes powers mod  $p$  (the period divides  $p-1$ ).
  - **Quadratic reciprocity** links residue information across primes.
  - The **CRT** combines information from several prime powers.
- We call this the **modulo spectrum** and expect it to make arithmetic patterns easier to learn. It exposes multiplicative structure **at depth 0**, before the network learns it across layers.

# What Does the Modulo Spectrum Reveal?

Powers of 2 and squares give repeating patterns modulo 3 and 5.

$n = 0, 1, \dots, 9$  in both examples

|          |   |   |   |   |    |    |    |     |     |     |              |
|----------|---|---|---|---|----|----|----|-----|-----|-----|--------------|
| $2^n$    | 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | ...          |
| Modulo 3 | 1 | 2 | 1 | 2 | 1  | 2  | 1  | 2   | 1   | 2   | ... Period 2 |
| Modulo 5 | 1 | 2 | 4 | 3 | 1  | 2  | 4  | 3   | 1   | 2   | ... Period 4 |

---

|          |   |   |   |   |    |    |    |    |    |    |              |
|----------|---|---|---|---|----|----|----|----|----|----|--------------|
| $n^2$    | 0 | 1 | 4 | 9 | 16 | 25 | 36 | 49 | 64 | 81 | ...          |
| Modulo 3 | 0 | 1 | 1 | 0 | 1  | 1  | 0  | 1  | 1  | 0  | ... Period 3 |
| Modulo 5 | 0 | 1 | 4 | 4 | 1  | 0  | 1  | 4  | 4  | 1  | ... Period 5 |

Modulo 3, powers of 2 repeat every 2 terms, while squares repeat every 3.

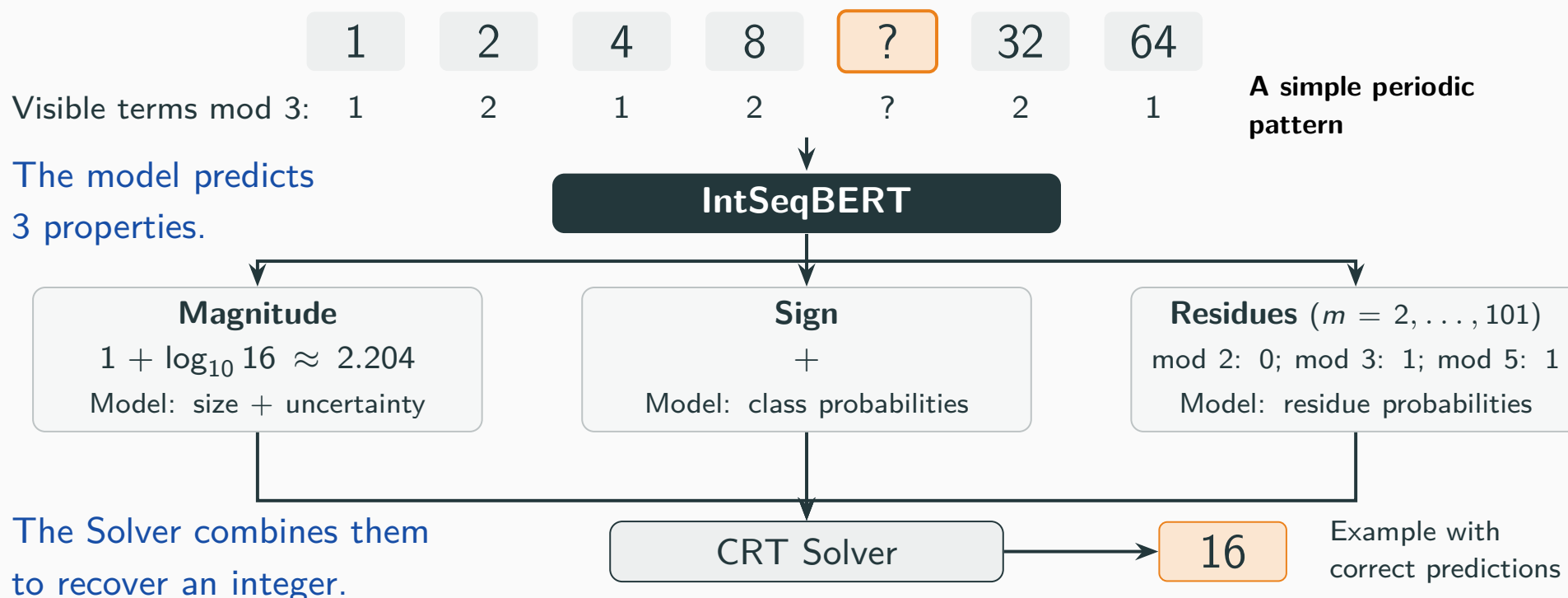
These different patterns give the model clues about each sequence's arithmetic structure.

## Task — Masked Sequence Modelling on OEIS

- To test this idea, we hide sequence terms and ask the model to predict them. This is **masked sequence modelling**: BERT-style learning for integers.
- We take up to the first **128 terms** and mask **15%** of the positions. The remaining terms provide the context.
- We also test next-term prediction, where the model sees only the preceding terms. Only the last term is masked in this setting.
- The **FACT** benchmark (NeurIPS 2022) tested a plain Transformer on 5 OEIS tasks. Unmasking was the hardest task. Our **Vanilla baseline** uses that architecture.
- For each hidden term, we predict its **magnitude, sign, and residues**.
  1. Magnitude measures size on a logarithmic scale.  $v_i = 1 + \log_{10} |x_i|$ .
  2. Sign is positive, negative, or 0.  $s_i \in \{+, -, 0\}$ .
  3. We predict residues for 100 moduli, from 2 to 101.  $x_i \bmod m$ ,  $m \in \{2, \dots, 101\}$ .

# A Worked Example — Predicting a Missing Integer

Here we hide 16 in the powers of 2 and use the surrounding terms as context.

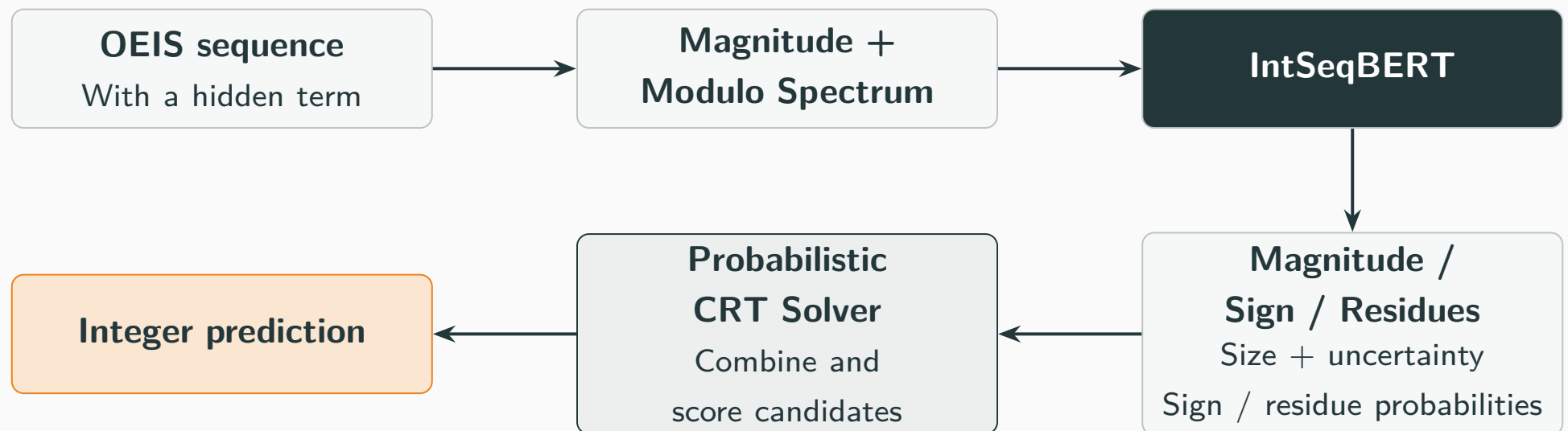


Here, correct predictions allow the Solver to recover 16.

**Training:** compare the 3 predictions with the hidden term's targets → update IntSeqBERT.

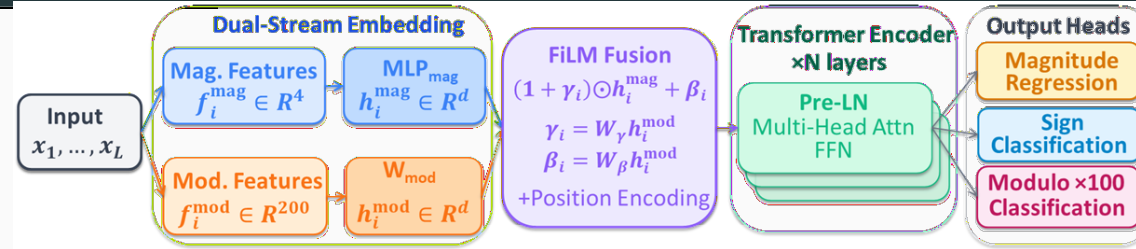
# The Whole Pipeline in 1 Slide

In the full system, the model predicts a missing term's properties, and the Solver recovers its value.



The Solver uses a **probabilistic** approach based on the Chinese Remainder Theorem, or CRT. It uses the predicted probabilities to score candidate integers.

# IntSeqBERT — Dual-Stream Architecture



- Let us look at the neural network architecture. One stream encodes magnitude and sign; the other encodes residues. The magnitude stream sends log-scale size and sign ( $\mathbb{R}^4$ ) through an MLP.
- The modulo stream places residues on a circle using sine and cosine. For the  $\mathbb{R}^{200}$  input,  $\phi_m(r) = [\sin(2\pi r/m), \cos(2\pi r/m)]$ . Residues  $m-1$  and 0 stay neighbours.
- We then combine these 2 streams and pass the result to a Transformer. With **FiLM**, the modulo stream scales and shifts the magnitude stream:  $\mathbf{e}_i = (1 + \gamma_i) \odot \mathbf{h}_i^{\text{mag}} + \beta_i$ .
- We test 3 model sizes: Small, Middle, and Large. Each uses a standard pre-LN Transformer encoder.

| Model size | Layers | Hidden dim. $d$ | Attention heads | Parameters |
|------------|--------|-----------------|-----------------|------------|
| Small      | 6      | 256             | 4               | 6.4M       |
| Middle     | 8      | 512             | 8               | 29.0M      |
| Large      | 12     | 768             | 12              | 91.5M      |

Model sizes describe network capacity; value buckets describe  $|x|$ .

# Training — Multi-Task Objective

- We train these models to predict all 3 outputs together, including uncertainty in magnitude.
  - The **magnitude head** predicts size and its **uncertainty**,  $(\mu_i, \sigma_i^2)$ . The Solver uses both.
  - The **sign head** predicts 1 of 3 classes.
  - The **modulo head** has 100 independent classifiers. Each loss is normalised by the maximum entropy  $\ln m$ , so mod 2 and mod 101 contribute fairly.
- We combine the losses with these weights:  $\mathcal{L} = \mathcal{L}_{\text{mag}} + \mathcal{L}_{\text{sign}} + 2 \mathcal{L}_{\text{mod}}$ .
- Training uses about 220,000 OEIS sequences over 200 epochs. The exact training count is **219,765**. We use **FP32** for training.
- All experiments run on a **single consumer GPU**. We use an RTX 3070 Ti with 8 GB, so other researchers can repeat the experiments.

# Neural-Symbolic Decoding — the CRT Solver

- The Solver first generates candidates in the predicted magnitude range. It searches the  $3\sigma$  interval, using its size to choose a method:
  - **Dense** ( $\Delta n \leq 10^6$ ) checks every integer in the interval.
  - **Sieve** ( $\leq 10^{14}$ ) uses CRT beam search with confident residues.
  - **CRT** ( $> 10^{14}$ ) generates large candidates directly with sparse CRT.
- Next, it scores each candidate using the magnitude estimate and all 100 residue distributions. For each  $m$ , it uses  $P_m(n \bmod m)$ .

**Probabilistic CRT: a weighted sum of log-probabilities.**

$$S(n) = \underbrace{-\frac{(v_n - \mu)^2}{2\sigma^2}}_{\text{magnitude log-likelihood}} + 0.3 \underbrace{\sum_{m=2}^{101} \ln P_m(n \bmod m)}_{\text{all modulo log-probabilities}}, \quad v_n = 1 + \log_{10} n$$

$P_m(r)$  is a residue probability. The weight 0.3 limits the effect of shared information.

- Finally, it returns the candidates with the highest scores. It takes the top  $k$  positive candidates and applies the sign; a zero sign returns 0.

## Dataset — 274,705 OEIS Sequences

- To evaluate the full system, we use about 70% of the OEIS, split into training, validation, and test sets. From 391,710 entries, we retain **274,705 (70.1%)** and split them 8:1:1.
- The excluded tags concern notation, task suitability, or data quality.
  - 5 tags depend on **how values are written**: cons, cofr, frac, base, word.
  - 2 tags **do not fit this task**: fini, tabl.
  - 4 are **quality labels**: dead, unkn, less, dumb.
- These choices define our task. The labels less/dumb are subjective, and tabl could be converted into sequences. Future benchmarks may choose differently. [▶ Tag definitions and examples](#)
- We compare performance by grouping terms by absolute value  $|x|$ . These value buckets are separate from model sizes.

| Value bucket | Range of each term $x$       |
|--------------|------------------------------|
| Small        | $0 \leq  x  < 10^2$          |
| Medium       | $10^2 \leq  x  < 10^5$       |
| Large        | $10^5 \leq  x  < 10^{20}$    |
| Huge         | $10^{20} \leq  x  < 10^{50}$ |
| Astronomical | $ x  \geq 10^{50}$           |

# Metrics — and 3 Questions

On the test data, we measure both the predicted properties and the final integer.

| Metric                                              | Definition                                                                                                                                                | What it measures                                      |
|-----------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------|
| <b>Magnitude Accuracy</b><br>( $\text{Acc}_{0.5}$ ) | Magnitude accuracy measures approximate size.<br>Within 0.5 on the log scale                                                                              | Approximate size; helps set the Solver's search range |
| <b>MMA</b><br>(mean modulo accuracy)                | Mean modulo accuracy, or MMA, measures residue prediction accuracy.<br>Averaged over all 100 moduli                                                       | Accuracy of residue predictions                       |
| <b>NIG</b><br>(normalised information gain)         | Normalised information gain, or NIG, measures the gain over uniform prediction.<br>$1 - \mathcal{L}_{\text{CE}}^{(m)} / \ln m$ ; 0 = no gain, 1 = perfect | Periodic structure learned for each modulus           |
| <b>Solver Top-<math>k</math></b>                    | Top- $k$ tests whether the correct integer is among the first $k$ candidates.                                                                             | Exact integer prediction                              |

For magnitude accuracy, an error within a factor of  $\sqrt{10} \approx 3.16$  is allowed.

True value  $10^6 \rightarrow$  predictions in about  $[3.2 \times 10^5, 3.2 \times 10^6]$  count as correct.

We use these metrics to test overall gains, gains by integer size, and exact prediction.

Q1: overall gains.    Q2: gains by value bucket.    Q3: exact prediction.

# What Should Happen If Our Idea Works?

Our baseline is **Vanilla**, a Transformer based on FACT.

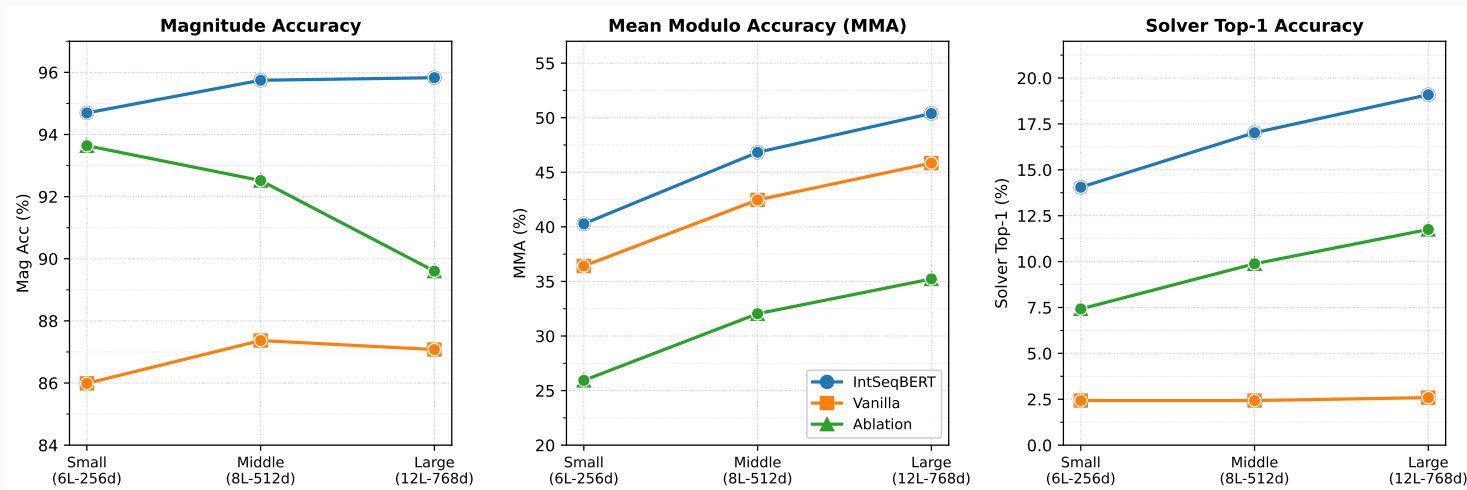
| Expected result                                                                                          | Where we test it               |
|----------------------------------------------------------------------------------------------------------|--------------------------------|
| 1 IntSeqBERT should outperform Vanilla.                                                                  | Q1: overall results            |
| 2 The advantage should be larger for large integers.<br>Ordinary tokenisation becomes problematic there. | Q2: value buckets              |
| 3 Learned residue representations should show number-theoretic structure.                                | NIG spectrum and totient ratio |

## Q1 — Does It Help? Yes, Consistently

| Model size | Model                | Mag Acc      | Sign Acc     | MMA          |
|------------|----------------------|--------------|--------------|--------------|
| Large      | <b>IntSeqBERT</b>    | <b>95.85</b> | <b>98.54</b> | <b>50.38</b> |
| Large      | Vanilla              | 86.97        | 97.66        | 45.85        |
| Large      | Ablation (no modulo) | 89.70        | 98.29        | 35.22        |

- The dual stream improves both magnitude and residue prediction over Vanilla. The Large model reaches **95.85%** magnitude accuracy (+**8.9 pt** over Vanilla) and **50.38%** MMA (+**4.5 pt**).
- For the Large model, removing the modulo stream lowers residue accuracy by about 15 percentage points. In the ablation, MMA falls by **15.2 pt** and magnitude accuracy by **6.2 pt**. Residues limit the possible integer values, helping both predictions.
- The same pattern holds for the Small and Middle models.

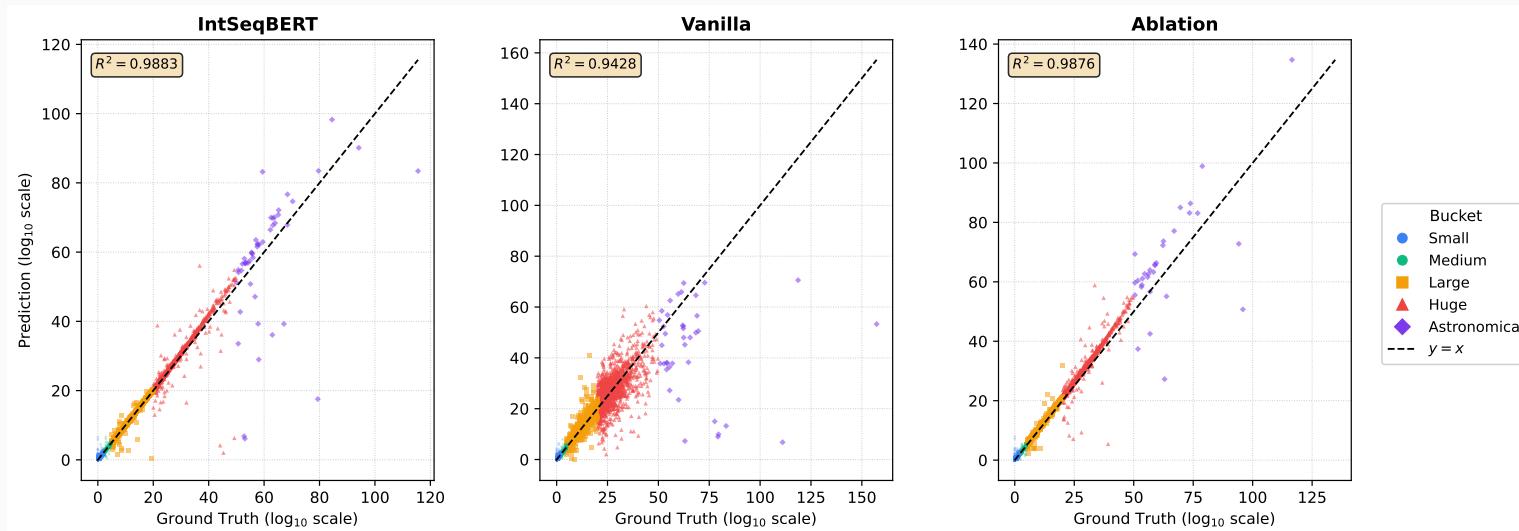
# Full Results & Scaling Behaviour



Model size: Small → Middle → Large

- For MMA, the ranking is always IntSeqBERT > Vanilla > Ablation.
- From Small to Large, residue accuracy gains about 10 percentage points; magnitude accuracy gains only 1 percentage point. From Small to Large, MMA gains **10.0 pt**; Mag Acc gains **1.1 pt**.
- Solver Top-1 also gains **5.0 pt**: 14.05 → 19.09.
- We see no clear sign of overfitting: Train 1.00 / Val 1.01 at epoch 200. We plan larger-scale retraining.

## Q2 — Where? Exactly Where Tokenisation Breaks



Prediction error (MSE)

| Value bucket | IntSeq       | Vanilla | Abl.         |
|--------------|--------------|---------|--------------|
| Small        | 0.111        | 0.138   | <b>0.103</b> |
| Large        | <b>0.162</b> | 2.100   | 0.381        |
| Huge         | <b>2.082</b> | 22.73   | 5.021        |

- Turning to integer size, Vanilla's mean squared magnitude error in the Large bucket is about 13 times ours. Vanilla uses UNK beyond its vocabulary. In the Large bucket, its MSE is **13×** ours. Overall  $R^2$  is **0.988** for IntSeqBERT and 0.943 for Vanilla.
- Overall scores can hide this gap because many sequences grow slowly. Many OEIS sequences grow slowly, so easier cases strongly affect those scores.

## Q3 — The Strictest Test: Exact Next-Term Prediction

| Model size | Model             | Top-1        | Top-10       |
|------------|-------------------|--------------|--------------|
| Large      | <b>IntSeqBERT</b> | <b>19.09</b> | <b>26.23</b> |
| Large      | Vanilla           | 2.59         | 3.80         |
| Large      | Ablation          | 11.75        | 21.79        |

- Next, we test whether the model can predict the exact next term. We use 10,000 sequences kept separate from training. For each sequence, we hide the last term and predict it from the earlier terms.
- The top-ranked answer is correct in **19.09%** of cases, versus **2.59%** for Vanilla. This is a **7.4-fold improvement**.
- Top-1 accuracy is **68.3%** in the Small value bucket and **20.8%** in Medium. Vanilla scores **0%** beyond its vocabulary.
- For the Large bucket and above, exact accuracy is near 0 at  $\text{MMA} \approx 50\%$ .

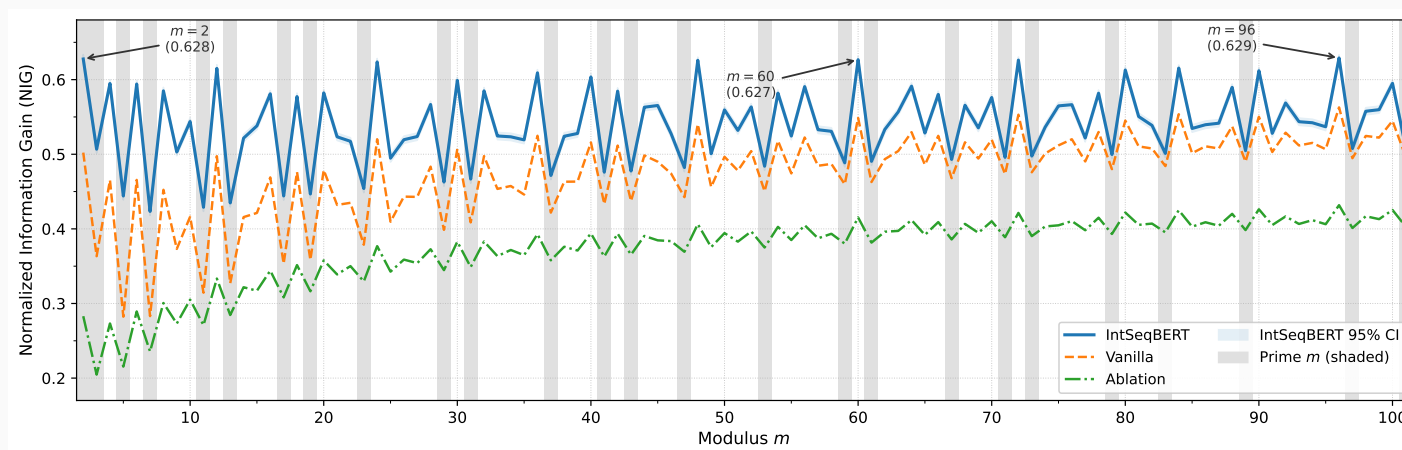
## What Does 19% Mean? — Better, but Still Low

- Despite this improvement, only about 1 in 5 predictions is exact.
- For large integers, exact accuracy is still **near 0**. This includes the Large, Huge, and Astronomical value buckets.

### **What this tells us**

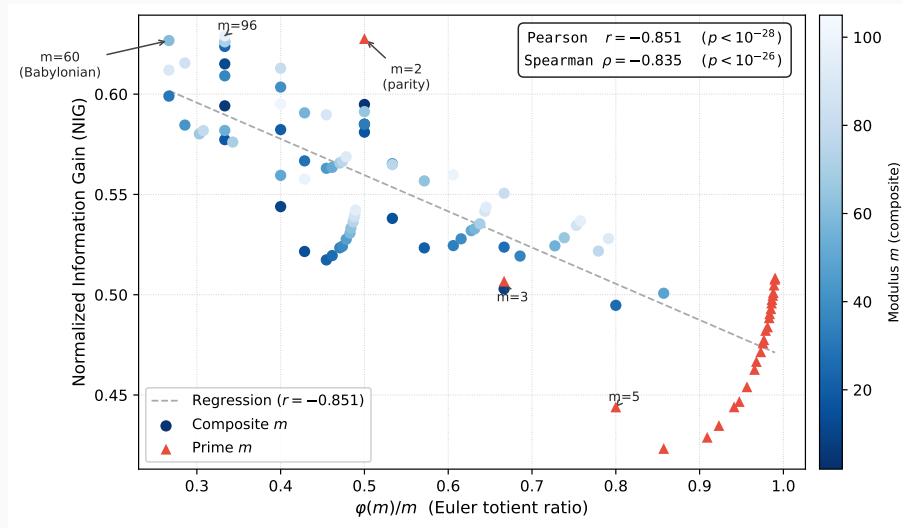
Recovering an exact integer from many uncertain residue predictions remains the main difficulty.

## But What Did the Network Learn? The NIG Spectrum



- We also want to understand the patterns the model has learned. This plot shows information gain for each modulus. NIG measures this for each modulus from 2 to 101, beyond overall accuracy.
- IntSeqBERT achieves higher information gain at every modulus, with a clear pattern of peaks. It is above both baselines across the **entire NIG spectrum**.
- The highest peak is at modulus 96, followed by modulus 2. This holds across all models and sizes:  $96 = 2^5 \times 3$  (NIG = 0.629). Mod 2 captures **parity**, which is common in the OEIS.
- Mod-2 accuracy is **85.7** for IntSeqBERT, 81.4 for Vanilla, and 72.1 for Ablation.

# Finding — Higher NIG with a Lower Totient Ratio



## Observation

The pattern is related to the factors of each modulus. We measure this using the totient ratio. A lower ratio tends to go with higher information gain.

The totient ratio is the fraction of residues coprime to  $m$ :

$$\phi(m)/m = \prod_{p|m} (1 - 1/p).$$

Pearson  $r = -0.851$  ( $p < 10^{-28}$ ).

This is a link found in our experiments, not a proven rule.

## Interpretation — CRT aggregation

A composite modulus may help by combining information from its factors.

For example,  $x \bmod 96$  gives  $x \bmod 2, 4, 8, 16, 32$  and  $x \bmod 3$  at the same time.

## Next steps

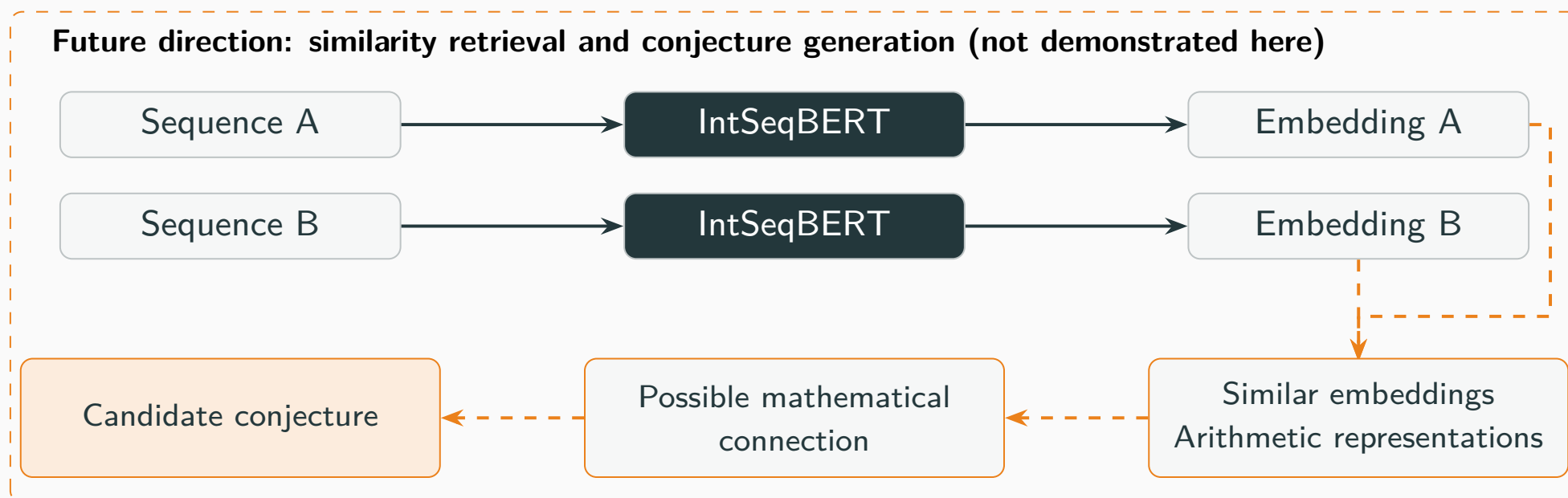
This suggests testing more composite moduli.

- Test **highly composite numbers beyond 101** before adding more primes.

# From Prediction to Mathematical Discovery

We now return to our goal: using these representations to find mathematical connections.

**This paper:** prediction improvement and learned arithmetic structure.



Those connections could suggest conjectures. This remains a goal for **future work**.

# Positioning in Computer Mathematics

- Finding related sequences could also help with program synthesis.
- Program synthesis seeks a rule that generates a sequence. Examples include **Alien Coding**, **QSynt**, and **Learning Conjecturing**.
- Related sequences could provide clues about which rules to try. The representations are learned across the whole corpus.
  - Synthesis asks: *“What program generates this sequence?”*
  - We ask: *“Which sequences are number-theoretically similar, even when they are generated in different ways?”*
- Adding this approach to systems such as Alien Coding might improve their accuracy.
- Our code, preprocessing, and evaluation are available at [github.com/aabaa/IntSeqBERT](https://github.com/aabaa/IntSeqBERT).

# Conclusion

1. The modulo spectrum makes arithmetic structure visible in the input. A **ring homomorphism** preserves multiplication **at depth 0**.
2. It improves prediction at every model size. Exact next-term accuracy improves by **7.4×** over Vanilla.
3. We observe a link between information gain and the totient ratio. NIG is negatively correlated with  $\varphi(m)/m$  ( $r = -0.851$ ).
4. Exact prediction remains difficult, especially for large integers. This supports further work on **neural-symbolic methods**.

We will improve decoding and explore how to find related sequences.

We plan approximate CRT for large integers, highly composite moduli beyond 101, larger-scale retraining, and **similarity retrieval & conjecture generation**.

Thank you for listening. — [github.com/aabaa/IntSeqBERT](https://github.com/aabaa/IntSeqBERT)

## Backup — Why Not Digit-Based Tokenisation?

- Another option is to generate integers 1 decimal digit at a time. Each digit is a token. We have not compared this approach experimentally in this paper.
- Our input gives the model residues directly. We target the **number-theoretic structure of the integer itself**. With digit tokens, the model must calculate residues from the digits.
- Digit-by-digit generation also changes the prediction task. It predicts a sequence of digits, while our model predicts 1 integer value.
- We plan to compare the 2 settings under the same conditions.

## Backup — Limitations

- Large integers remain difficult because residue predictions are uncertain. In **13.4%** of Solver calls, no valid candidate is found (`none` mode). Approximate CRT or less strict residue constraints may help.
- The dataset also affects our overall scores. Many nonn sequences help sign accuracy; many slowly growing sequences help magnitude accuracy.
- Our experiments use limited compute and 1 random seed. We use 1 **8 GB GPU**, seed **42**, and a vocabulary of **20,003** tokens. Vanilla uses the *FACT architecture* with our settings; we do not reproduce FACT's reported results.

## Backup — How Hard Is Multiplication for Neural Networks?

- The difficulty of multiplication depends on the model and the training method. In **circuit theory**,  $n$ -bit multiplication is not in  $AC^0$  (constant-depth AND/OR). Polynomial-size threshold circuits can do it with **3 layers**, and 3 is optimal. Iterated products need only 4 layers (Siu & Roychowdhury, *SIAM J. Discrete Math.*, 1994).
- Log-precision **Transformers** are upper-bounded by uniform  $TC^0$  (Merrill & Sabharwal 2023). This connects multiplication to model limits.
- Experiments show that learning multiplication can still be difficult. GPT-4 zero-shot accuracy falls from **59%** for  $3 \times 3$ -digit products to **4%** for  $4 \times 4$ -digit products (Dziri+, *Faith and Fate*, NeurIPS 2023).
- A **2-layer Transformer** on modular arithmetic generalises only after a very long training period: **grokking** (Power+ 2022, arXiv:2201.02177).
- Shen et al. (2023) report **84%** accuracy on  $15 \times 15$ -digit multiplication with a **12-layer Transformer**, zero-padding, and reversed output digits (arXiv:2311.14737, Sec. 2 / Table 5). This does not establish a minimum layer count.
- Bai et al. (2025) study a **2-layer** model on  $4 \times 4$ -digit multiplication: implicit-CoT training succeeds where standard fine-tuning fails. Their analysis finds a tree-like attention mechanism (arXiv:2510.00184).

## Backup — OEIS Tags: Number Representations

These tags describe digits, fractions, or language-based rules. We leave them outside the scope of this study.

| Tag  | Meaning                                                | Example                                                                        |
|------|--------------------------------------------------------|--------------------------------------------------------------------------------|
| cons | Digits of a constant, usually in base 10.              | $\pi = 3.14159\dots$ gives 3, 1, 4, 1, 5, 9, $\dots$                           |
| cofr | Coefficients in a continued fraction.                  | $\pi = [3; 7, 15, 1, 292, \dots]$ gives 3, 7, 15, 1, 292, $\dots$              |
| frac | Numerators or denominators of a sequence of fractions. | For $1/2, 2/3, 3/4, \dots$ , the denominators are 2, 3, 4, $\dots$             |
| base | A rule that depends on the number base.                | Sum of decimal digits: $19 \mapsto 1 + 9 = 10$ .                               |
| word | A rule that depends on words in a language.            | Number of letters in English number names: “one”, “two”, “three” give 3, 3, 5. |

These sequences can have rich mathematical structure. Excluding them is a choice about our task.

Source: OEIS keyword definitions. Examples illustrate each type.

## Backup — OEIS Tags: Task Scope and Data Selection

We also exclude finite sequences, tables, and entries with the following labels.

| Tag  | Meaning                                      | Why we exclude it / what to keep in mind                                           |
|------|----------------------------------------------|------------------------------------------------------------------------------------|
| fini | A sequence known to be finite.               | There is no next term after its endpoint.                                          |
| tabl | A regular table or triangle read row by row. | For example, Pascal's triangle. Row boundaries add structure beyond a simple list. |
| dead | An incorrect or duplicate entry.             | We exclude known errors and duplicates.                                            |
| unkn | The definition or context is unknown.        | We exclude entries whose meaning is unclear.                                       |
| less | An entry judged less interesting.            | A subjective OEIS label; it does not mean the sequence is incorrect.               |
| dumb | An entry judged unimportant.                 | Also a subjective label, not a mathematical test.                                  |

These filters define our dataset. Other studies may include these sequences.

Source: OEIS keyword definitions. Selection reasons describe our study.